

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Лабораторная номер 3
по дисциплине Фронтенд разработка

Выполнил работу студент:
Воронин Илья Андреевич
ИСУ: 372736
Группа: J3212

Санкт-Петербург, 2026

Задача

В рамках лабораторной работы № 3 необходимо мигрировать ранее разработанное приложение (ЛР1, ЛР2) на фреймворк Vue.js. Согласно методическим требованиям, итоговое SPA должно удовлетворять следующим критериям:

- подключён клиентский маршрутизатор (**vue-router**);
- реализовано взаимодействие с внешним API, желательно через **axios**;
- проведено разумное деление приложения на компоненты;
- повторяющаяся логика вынесена в **composable**-функции.

Выбранный на курсе вариант - № 9 «Сервис для управления проектами и командной работы» (далее - ProjectHub). Приложение содержит обязательные страницы: вход, регистрацию, личный кабинет, страницу поиска с фильтрацией, страницу проекта (канбан-доска, сроки, обсуждения) и роли участников команды.

Ход работы

2.1. Используемый стек технологий

В качестве сборщика и dev-сервера выбран **Vite** - он обеспечивает быструю горячую перезагрузку и нативно поддерживает Vue SFC. Список ключевых зависимостей приведён в `package.json`:

Листинг 1: Зависимости проекта (`package.json`)

```
1 {
2   "name": "projecthub-lab3",
3   "type": "module",
4   "scripts": {
5     "dev": "vite",
6     "build": "vite build",
7     "api": "node server.cjs"
8   },
9   "dependencies": {
10    "axios": "^1.7.9",
11    "bootstrap": "^5.3.2",
12    "vue": "^3.4.21",
13    "vue-router": "^4.3.0"
14  },
15  "devDependencies": {
16    "@vitejs/plugin-vue": "^5.0.4",
17    "json-server": "^0.17.4",
18    "vite": "^5.2.0"
19  }
20 }
```

В качестве UI-фреймворка переиспользован Bootstrap 5 (для сохранения визуальной преемственности с ЛР1/ЛР2). Для всех HTTP-запросов используется `axios`. Для имитации backend-а развёрнут `json-server`, поверх которого написан небольшой кастомный middleware (`server.cjs`) с эмуляцией авторизации по JWT-подобному токenu.

2.2. Структура проекта

Проект разбит на типичные для Vue-приложения директории:

Листинг 2: Структура директории `src/`

```
1 src/
2   api/           - модули-обертки над axios (auth, projects, tasks, ...)
3   components/    - переиспользуемые UI-компоненты
4   composables/   - функции с переиспользуемой логикой
5   router/        - конфигурация vue-router
6   views/         - страницы приложения, привязанные к маршрутам
7   assets/        - глобальные стили
8   App.vue        - корневой компонент приложения
9   main.js        - точка входа: создание приложения, монтирование
10  api/db.json     - состояние мокового API
11  server.cjs      - express-обертка над json-server с авторизацией
```

2.3. Точка входа и корневой компонент

В `main.js` создаётся экземпляр приложения, к нему подключается маршрутизатор и корневой компонент `App.vue`. Сам `App.vue` содержит шапку, область `<RouterView/>` и подвал, скрывающийся на страницах входа/регистрации через мета-флаг маршрута:

Листинг 3: `src/App.vue` - корневой компонент

```
1 <script setup>
2 import { computed } from 'vue';
3 import { useRoute } from 'vue-router';
4 import AppNavbar from './components/AppNavbar.vue';
5 import AppFooter from './components/AppFooter.vue';
6
7 const route = useRoute();
8 const showFooter = computed(() => route.meta.hideFooter !== true);
9 </script>
10
11 <template>
12   <AppNavbar />
13   <main class="flex-grow-1">
14     <RouterView />
15   </main>
16   <AppFooter v-if="showFooter" />
17 </template>
```

2.4. Маршрутизация: navigation guards и meta

В роутере объявлены маршруты для всех экранов приложения. Каждому маршруту прописана мета-информация: заголовок вкладки (`title`), требование авторизации (`requiresAuth`), флаг «только для гостей» (`guestOnly`) и признак скрытия подвала (`hideFooter`). Глобальный guard `beforeEach` перенаправляет неавторизованных пользователей на страницу входа, а `afterEach` обновляет `document.title`.

Листинг 4: `src/router/index.js`

```
1 import { createRouter, createWebHistory } from 'vue-router';
2 import { isAuthenticated } from '../composables/useAuth';
3
4 const routes = [
5   { path: '/', name: 'home',
6     component: () => import('../views/HomeView.vue'),
7     meta: { title: 'ProjectHub' } },
8   { path: '/login', name: 'login',
9     component: () => import('../views/LoginView.vue'),
10    meta: { title: 'Vhod', guestOnly: true, hideFooter: true } },
11   { path: '/register', name: 'register',
12     component: () => import('../views/RegisterView.vue'),
13     meta: { title: 'Registratsia', guestOnly: true, hideFooter: true } },
14   { path: '/dashboard', name: 'dashboard',
15     component: () => import('../views/DashboardView.vue'),
16     meta: { title: 'Lichnyi kabinet', requiresAuth: true } },
17   { path: '/search', name: 'search',
18     component: () => import('../views/SearchView.vue'),
19     meta: { title: 'Poisk', requiresAuth: true } },
20   { path: '/project/:id', name: 'project',
21     component: () => import('../views/ProjectView.vue'),
22     props: true,
23     meta: { title: 'Proekt', requiresAuth: true } },
24   { path: '/*', redirect: '/' },
25 ];
26
27 const router = createRouter({
28   history: createWebHistory(),
29   routes,
30 });
31
32 router.beforeEach((to) => {
33   if (to.meta.requiresAuth && !isAuthenticated()) {
34     return { name: 'login', query: { redirect: to.fullPath } };
35   }
36   if (to.meta.guestOnly && isAuthenticated()) {
37     return { name: 'dashboard' };
38   }
39   return true;
```

```

40 });
41
42 router.afterEach((to) => {
43   if (to.meta.title) document.title = to.meta.title;
44 });
45
46 export default router;

```

Для всех страниц, требующих авторизации, используется ленивая загрузка `((() => import(...)))`, что уменьшает размер первоначального бандла. Маршрут `/project/:id` объявлен с `props: true` - это позволяет получать идентификатор проекта в компонент в виде входного параметра, без обращения к `useRoute()`.

2.5. Composable-функции

Повторяющаяся логика была вынесена в четыре composable-модуля:

- **useAuth** - хранит и реактивно отдаёт текущего пользователя и его токен, синхронизирует состояние с `localStorage`;
- **useUsers** - кэширует список пользователей системы (нужен для отображения исполнителей и аватаров);
- **useTaskMeta** - словари статусов и приоритетов задач, соответствующие им цвета и переводы для канбан-карточек;
- **useI18n** - легковесная локализация интерфейса.

В качестве примера приведён **useAuth** - именно он переиспользуется и в `App.vue`, и в `guard-ax` роутера:

Листинг 5: `src/composables/useAuth.js`

```

1 import { ref, computed, readonly } from 'vue';
2
3 const AUTH_USER_KEY = 'projecthub_user';
4 const AUTH_TOKEN_KEY = 'projecthub_token';
5
6 function safeParse(raw) {
7   if (!raw) return null;
8   try { return JSON.parse(raw); } catch (e) { return null; }
9 }
10
11 const user = ref(safeParse(localStorage.getItem(AUTH_USER_KEY)));
12 const token = ref(localStorage.getItem(AUTH_TOKEN_KEY));
13
14 export function getToken() { return token.value; }
15 export function isAuthenticated() {
16   return !!user.value && !!token.value;

```

```

17 }
18
19 export function setAuth(nextUser, nextToken) {
20   user.value = nextUser || null;
21   token.value = nextToken || null;
22   if (nextUser) localStorage.setItem(AUTH_USER_KEY, JSON.stringify(nextUser));
23   else localStorage.removeItem(AUTH_USER_KEY);
24   if (nextToken) localStorage.setItem(AUTH_TOKEN_KEY, nextToken);
25   else localStorage.removeItem(AUTH_TOKEN_KEY);
26 }
27
28 export function clearAuth() { setAuth(null, null); }
29
30 export function useAuth() {
31   return {
32     user: readonly(user),
33     isAuthenticated: computed(() => !!user.value && !!token.value),
34     setAuth,
35     logout: clearAuth,
36   };
37 }

```

Состояние объявлено на уровне модуля (`user`, `token`), поэтому все компоненты, импортирующие `useAuth`, разделяют один и тот же реактивный источник истины - по сути это лёгкая замена `Vuex`/`Pinia` для небольшого приложения.

2.6. Деление на компоненты

Каждая страница (`view`) собирается из мелких компонентов, отвечающих за одну зону UI. Это упрощает повторное использование и сопровождение.

- **Слой страниц** (`views/`): `HomeView`, `LoginView`, `RegisterView`, `DashboardView`, `SearchView`, `ProjectView`.
- **Каркас** (`components/`): `AppNavbar`, `AppFooter`.
- **Канбан и задачи**: `KanbanBoard` (колонки), `KanbanCard` (карточка задачи), `TaskListItem` (строчное представление), `TaskDetailsModal` (модалка с деталями).
- **Проекты и роли**: `ProjectCard`, `AddProjectModal`, `AddTaskModal`, `RolesModal`.
- **Прочее**: `DiscussionList` (обсуждения), `DeadlinesTable` (ближайшие дедлайны), `Notification` (уведомления в шапке).

Связь «родитель–потомок» построена через `props/emits`, без сквозного проброса состояния.

2.7. Работа с API через axios

Точка взаимодействия с сервером - единый axios-клиент в `src/api/client.js`. К каждому запросу автоматически добавляется токен авторизации (через `interceptor`). Конкретные эндпоинты обёрнуты в небольшие модули по доменам: `auth.js`, `projects.js`, `tasks.js`, `users.js`, `discussions.js`, `notifications.js`. Это позволяет компонентам не знать о деталях URL-схемы.

Листинг 6: Использование API в `LoginView.vue`

```
1 import { login } from '../api/auth';
2 import { useAuth } from '../composables/useAuth';
3
4 const { setAuth } = useAuth();
5
6 async function onSubmit() {
7   error.value = '';
8   submitting.value = true;
9   try {
10     const data = await login(email.value.trim(), password.value);
11     if (!data || !data.user) throw new Error('Oshibka otveta servera');
12     setAuth(data.user, data.token);
13     const redirect = typeof route.query.redirect === 'string'
14       ? route.query.redirect : null;
15     router.push(redirect || { name: 'dashboard' });
16   } catch (e) {
17     error.value = (e?.response?.data?.message)
18       || e.message || 'Oshibka vhoda';
19   } finally {
20     submitting.value = false;
21   }
22 }
```

Вывод

В ходе выполнения лабораторной работы приложение ProjectHub было успешно мигрировано с многостраничного HTML/JS на одностраничное приложение на Vue.js 3 с использованием Composition API и SFC-компонентов. Все поставленные требования выполнены:

- **Маршрутизация.** Подключён `vue-router 4` в режиме `createWebHistory`; настроены ленивая загрузка страниц, `meta`-поля (`title`, `requiresAuth`, `guestOnly`, `hideFooter`) и `navigation guards` (`beforeEach`, `afterEach`).
- **Внешний API.** Все запросы идут через единый `axios`-клиент с `interceptor`-ом, прокидывающим токен авторизации; эндпоинты разнесены по доменным модулям.

- **Компонентный подход.** Приложение разбито на 6 страниц-views и 13 переиспользуемых компонентов, общение между ними построено через `props/emits`.
- **Composables.** Сквозная логика (авторизация, словари пользователей, словари задач, локализация) вынесена в четыре composable-функции, переиспользуемых и компонентами, и navigation guards.

В результате получено компактное, типизированно-читаемое и легко расширяемое приложение, в котором добавление новой страницы или домена данных сводится к созданию нового view, маршрута и тонкого API-модуля. Закреплены навыки работы с фреймворком Vue.js, клиентским роутингом, composable-функциями и асинхронным взаимодействием с REST API через axios.